

Exercise 1a: Forward Kinematics of the ABB IRB 120

Prof. Marco Hutter*

Teaching Assistants:

Arjun Bhardwaj, Thomas Li, Ennio Schnieder

September 23, 2026



Figure 1: The ABB IRW 120 robot arm.

Abstract

The aim of this exercise is to calculate the forward and inverse kinematics of an ABB robot arm. In doing so, you will practice the use of different representations of the end-effector's orientation as well as how to check whether your implementations are correct. Essentially, the task is to implement the functions for computing the forward and inverse kinematics using symbolic and numerical computations in MATLAB. A separate MATLAB script will be provided for the 3D visualization of the robot arm.

*original contributors include Michael Blösch, Dario Bellicoso, and Samuel Bachmann

1 Introduction

The following exercise is based on an ABB IRB 120 depicted in Figure 2. It is a 6-link robotic manipulator with a fixed base. During the exercise you will implement several different MATLAB functions, which you should test carefully since the following tasks are often dependent on them. To help you with this, use the provided script prototypes (download from Moodle).

2 Forward Kinematics

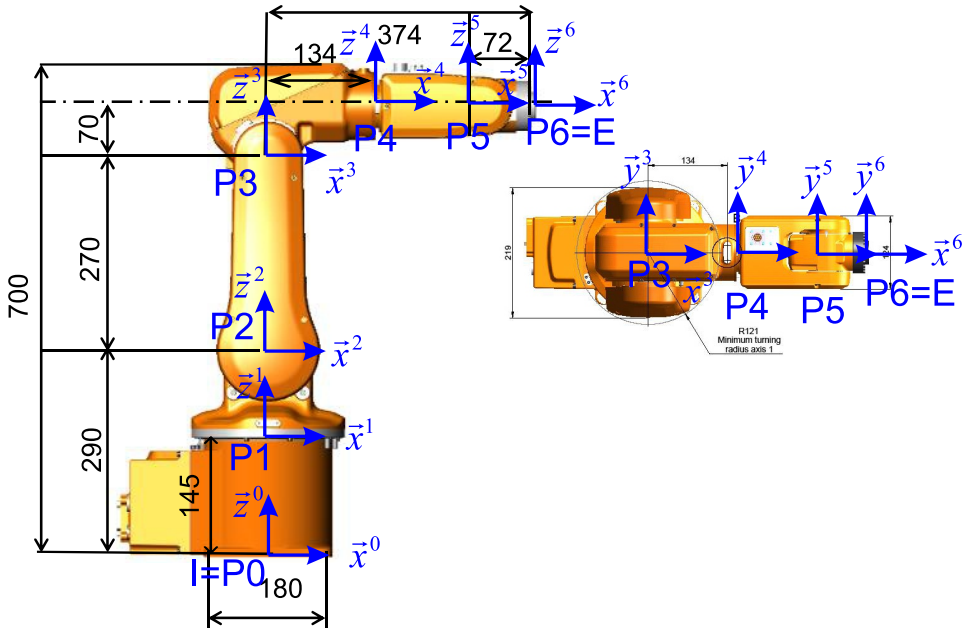


Figure 2: ABB IRB 120 with coordinate systems and joints. The units are mm.

Throughout this document, we will employ I for denoting the inertial world coordinate system (coordinate system $P0$ in Figure 2) and E for the coordinate system attached to the end-effector (coordinate system $P6$ in Figure 2).

You should always check your solutions with the provided script `evaluate_problems.m`. This script compares your implementation with our solution on random data points.

Exercise 2.1

Define a vector $\mathbf{q}$ of generalized coordinates to describe the configuration of the ABB IRB120. Recall that generalized coordinates should be *complete* and *independent*. The former property means that they should fully describe the configuration of the robot while at the same time comprising a minimal set of coordinates. The latter property refers to the fact that the each generalized coordinate must not be a function of any of the others.

Solution 2.1

The generalized coordinates can be chosen as the single joint angles between subsequent links. Any other *complete* and *independent* linear combination of the single joint angles is also a valid solution.

$$\mathbf{q} = (q_1, \dots, q_6)^T \in \mathbb{R}^6 \quad (1)$$

Exercise 2.2

Assume from here on that the generalized coordinates $\mathbf{q}$ are the joint angles of the robot arm numbered according to Figure 2. Positive angles imply rotations around the positive coordinate axis.

Compute the homogeneous transformations matrices $\mathbf{T}_{k-1,k}(q_k)$, $\forall k = 1, \dots, 6$. Additionally, find the constant homogeneous transformations between the inertial frame and frame 0 ($\mathbf{T}_{I0}$) and between frame 6 and the end-effector frame ($\mathbf{T}_{6E}$). Please implement the following functions (i.e., replace the zero assignments with your solution):

```
1 function TI0 = getTransformI0()
2 % Input: void
3 % Output: homogeneous transformation Matrix from frame 0 to the ...
   inertial frame I. T_I0
4
5 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
6 TI0 = zeros(4);
7 end
8
9 function T01 = jointToTransform01(q)
10 % Input: joint angles
11 % Output: homogeneous transformation Matrix from frame 1 to frame ...
   0. T_01
12
13 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
14 T01 = zeros(4);
15 end
16
17 function T12 = jointToTransform12(q)
18 % Input: joint angles
19 % Output: homogeneous transformation Matrix from frame 2 to frame ...
   1. T_12
20
21 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
22 T12 = zeros(4);
23 end
24
25 function T23 = jointToTransform23(q)
26 % Input: joint angles
27 % Output: homogeneous transformation Matrix from frame 3 to frame ...
   2. T_23
28
29 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
30 T23 = zeros(4);
31 end
32
33 function T34 = jointToTransform34(q)
34 % Input: joint angles
35 % Output: homogeneous transformation Matrix from frame 4 to frame ...
   3. T_34
36
37 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
38 T34 = zeros(4);
39 end
40
41
42 function T45 = jointToTransform45(q)
43 % Input: joint angles
44 % Output: homogeneous transformation Matrix from frame 5 to frame ...
   4. T_45
45
46 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
47 T45 = zeros(4);
```

```

48 end
49
50 function T56 = jointToTransform56(q)
51     % Input: joint angles
52     % Output: homogeneous transformation Matrix from frame 6 to frame ...
53     5. T_56
54
55     % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
56     T56 = zeros(4);
57 end
58
59 function T6E = getTransform6E()
60     % Input: void
61     % Output: homogeneous transformation Matrix from the end-effector ...
62     frame E to frame 6. T_6E
63
64     % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
65     T6E = zeros(4);
66 end

```

Solution 2.2

Remember that a homogeneous transformation matrix is expressed in the form

$$\mathbf{T}_{hk}(q_k) = \begin{bmatrix} \mathbf{C}_{hk}(q_k) & {}_h\mathbf{r}_{hk}(q_k) \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}. \quad (2)$$

For the ABB IRB 120, each $\mathbf{T}_{hk}(q_k)$ is composed by an elementary rotation a single joint axis and a translation defined by the manipulator kinematic parameters. By defining the elementary rotations matrices about each axis as

$$\mathbf{C}_z(\varphi) = \begin{bmatrix} \cos(\varphi) & -\sin(\varphi) & 0 \\ \sin(\varphi) & \cos(\varphi) & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (3)$$

$$\mathbf{C}_y(\varphi) = \begin{bmatrix} \cos(\varphi) & 0 & \sin(\varphi) \\ 0 & 1 & 0 \\ -\sin(\varphi) & 0 & \cos(\varphi) \end{bmatrix} \quad (4)$$

$$\mathbf{C}_x(\varphi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos(\varphi) & -\sin(\varphi) \\ 0 & \sin(\varphi) & \cos(\varphi) \end{bmatrix}, \quad (5)$$

one can write

$$\mathbf{T}_{01}(q_1) = \begin{bmatrix} \mathbf{C}_z(q_1) & {}_0\mathbf{r}_{01}(q_1) \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \quad (6)$$

$$\mathbf{T}_{12}(\theta_2) = \begin{bmatrix} \mathbf{C}_y(q_2) & {}_1\mathbf{r}_{12}(q_2) \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \quad (7)$$

$$\mathbf{T}_{23}(q_3) = \begin{bmatrix} \mathbf{C}_y(q_3) & {}_2\mathbf{r}_{23}(q_3) \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \quad (8)$$

$$\mathbf{T}_{34}(q_4) = \begin{bmatrix} \mathbf{C}_x(q_4) & {}_3\mathbf{r}_{34}(q_4) \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \quad (9)$$

$$\mathbf{T}_{45}(q_5) = \begin{bmatrix} \mathbf{C}_y(q_5) & {}_4\mathbf{r}_{45}(q_5) \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \quad (10)$$

$$\mathbf{T}_{56}(q_6) = \begin{bmatrix} \mathbf{C}_x(q_6) & {}_5\mathbf{r}_{56}(q_6) \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}. \quad (11)$$

Finally, the constant homogeneous transformations $\mathbf{T}_{I0}$ and $\mathbf{T}_{6E}$ are simply the identity matrix $\mathbf{I}_{4 \times 4}$.

```

1 function TI0 = getTransformI0()
2 % Input: void
3 % Output: homogeneous transformation Matrix from the inertial ...
4 % frame 0 to frame I. T_I0
5 TI0 = eye(4);
6 end
7 function T01 = jointToTransform01(q)
8 % Input: joint angle
9 % Output: homogeneous transformation Matrix from frame 1 to frame ...
10 % 0. T_01
11 if (length(q)>1)
12 q = q(1);
13 end
14 T01 = [cos(q), -sin(q), 0, 0;
15 sin(q), cos(q), 0, 0;
16 0, 0, 1, 0.145;
17 0, 0, 0, 1];
18 end
19 function T12 = jointToTransform12(q)
20 % Input: joint angle
21 % Output: homogeneous transformation Matrix from frame 2 to frame ...
22 % 1. T_12
23 if (length(q)>1)
24 q = q(2);
25 end
26 T12 = [cos(q), 0, sin(q), 0;
27 0, 1, 0, 0;
28 -sin(q), 0, cos(q), 0.145;
29 0, 0, 0, 1];
30 end
31 function T23 = jointToTransform23(q)
32 % Input: joint angle
33 % Output: homogeneous transformation Matrix from frame 3 to frame ...
34 % 2. T_23
35 if (length(q)>1)
36 q = q(3);
37 end
38 T23 = [cos(q), 0, sin(q), 0;
39 0, 1, 0, 0;
40 -sin(q), 0, cos(q), 0.270;
41 0, 0, 0, 1];
42 end
43 function T34 = jointToTransform34(q)
44 % Input: joint angle
45 % Output: homogeneous transformation Matrix from frame 4 to frame ...
46 % 3. T_34
47 if (length(q)>1)
48 q = q(4);
49 end
50 T34 = [1, 0, 0, 0.134;
51 0, cos(q), -sin(q), 0;
52 0, sin(q), cos(q), 0.070;
53 0, 0, 0, 1];
54 end
55 function T45 = jointToTransform45(q)
56 % Input: joint angle
57 % Output: homogeneous transformation Matrix from frame 5 to frame ...
58 % 4. T_45
59 if (length(q)>1)
60 q = q(5);
61 end
62 T45 = [cos(q), 0, sin(q), 0.168;

```

```

62         0,    1,    0,    0;
63        -sin(q), 0,   cos(q), 0;
64         0,    0,    0,    1];
65 end
66
67 function T56 = jointToTransform56(q)
68 % Input: joint angle
69 % Output: homogeneous transformation Matrix from frame 5 to frame ...
70         6. T56
71 if (length(q)>1)
72     q = q(6);
73 end
74 T56 = [1,    0,    0,    0.072;
75        0,   cos(q), -sin(q), 0;
76        0,   sin(q),  cos(q), 0;
77        0,    0,    0,    1];
78
79 function T6E = getTransform6E()
80 % Input: void
81 % Output: homogeneous transformation Matrix from the end-effector ...
82         frame E to frame 6. T6E
83 T6E = eye(4);
84 end

```

Exercise 2.3

Find the end-effector position vector ${}^I\mathbf{r}_{IE} = {}^I\mathbf{r}_{IE}(\mathbf{q})$. Please implement the following function:

```

1 function I_r_IE = jointToPosition(q)
2 % Input: joint angles
3 % Output: position of end-effector w.r.t. inertial frame. I_r_IE
4
5 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
6 I_r_IE = zeros(3,1);
7 end

```

Solution 2.3

The end-effector position is given by the direct kinematics, represented in matrix form by the homogeneous transformation $\mathbf{T}_{IE}(\mathbf{q})$, which can be found by successive concatenation of coordinate frame transformations.

$$\mathbf{T}_{IE}(\mathbf{q}) = \mathbf{T}_{I0} \cdot \left(\prod_{k=1}^6 \mathbf{T}_{k-1,k} \right) \cdot \mathbf{T}_{6E} = \begin{bmatrix} \mathbf{C}_{IE} & {}^I\mathbf{r}_{IE}(\mathbf{q}) \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \quad (12)$$

The end-effector position can then be found by selecting the fourth column of $\mathbf{T}_{IE}(\mathbf{q})$.

$$\begin{bmatrix} \mathbf{r}(\mathbf{q}) \\ 1 \end{bmatrix} = \mathbf{T}_{IE}(\mathbf{q}) \cdot \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad (13)$$

```

1 function I_r_IE = jointToPosition(q)
2 % Input: joint angles
3 % Output: position of end-effector w.r.t. inertial frame. I_r_IE
4 T_IE = getTransformI0() * ...
5         jointToTransform01(q) * ...
6         jointToTransform12(q) * ...

```

```

7         jointToTransform23(q)*...
8         jointToTransform34(q)*...
9         jointToTransform45(q)*...
10        jointToTransform56(q)*...
11        getTransform6E();
12    I_r_IE = T_IE(1:3,4);
13 end

```

Exercise 2.4

What is the end-effector position for $\mathbf{q} = \begin{pmatrix} \pi/6 \\ \pi/6 \\ \pi/6 \\ \pi/6 \\ \pi/6 \\ \pi/6 \end{pmatrix}$?

Use Matlab (`abbRobot.setJointPositions(q)`) to visualize it.

Solution 2.4

From the direct kinematics equations found earlier, it is:

$${}^I\mathbf{r}_{IE} = {}^I\mathbf{r}_{IE}(\mathbf{q}) = \begin{bmatrix} 0.2948 \\ 0.1910 \\ 0.2277 \end{bmatrix}. \quad (14)$$

Exercise 2.5

Find the end-effector rotation matrix $\mathbf{C}_{IE} = \mathbf{C}_{IE}(\mathbf{q})$. Please implement the following function:

```

1 function C_IE = jointToRotMat(q)
2 % Input: joint angles
3 % Output: rotation matrix which projects a vector defined in the
4 % end-effector frame E to the inertial frame I, C_IE.
5
6 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
7 C_IE = zeros(3);
8 end

```

Solution 2.5

From the structure of the direct kinematics equations found earlier, it follows that the end-effector rotation matrix is obtained by extracting the first three rows and the first three columns from $\mathbf{T}_{IE}(\mathbf{q})$. This operation can be compactly written in matrix form:

$$\mathbf{C}_{IE}(\mathbf{q}) = \begin{bmatrix} \mathbf{I}_{3 \times 3} & \mathbf{0}_{3 \times 1} \end{bmatrix} \cdot \mathbf{T}_{IE}(\mathbf{q}) \cdot \begin{bmatrix} \mathbf{I}_{3 \times 3} \\ \mathbf{0}_{1 \times 3} \end{bmatrix}. \quad (15)$$

```

1 function C_IE = jointToRotMat(q)
2 % Input: joint angles
3 % Output: rotation matrix which projects a vector defined in the
4 % end-effector frame E to the inertial frame I, C_IE.
5 T_IE = getTransformI0() * ...
6         jointToTransform01(q) * ...
7         jointToTransform12(q) * ...
8         jointToTransform23(q) * ...
9         jointToTransform34(q) * ...
10        jointToTransform45(q) * ...

```

```

11         jointToTransform56(q) * ...
12         getTransform6E();
13     C_IE = T_IE(1:3,1:3);
14 end

```

Exercise 2.6

Find the quaternion representing the attitude of the end-effector $\xi_{IE} = \xi_{IE}(\mathbf{q})$. Please also implement the following function:

- Two functions for converting from quaternion to rotation matrices and vice-versa. Test these by converting from quaternions to rotation matrices and back to quaternions.
- The quaternion multiplication $\mathbf{q} \otimes \mathbf{p}$
- The passive rotation of a vector with a given quaternion. This can be implemented in different ways which can be tested with respect to each other. The easiest way is to transform the quaternion to the corresponding rotation matrix (by using the function from above) and then multiply the matrix with the vector to be rotated.

Also check that your two representations for the end-effector orientation match with each other. In total you should write the following five functions:

```

1  function quat = jointToQuat(q)
2  % Input: joint angles
3  % Output: quaternion representing the orientation of the end-effector
4  % q_IE.
5
6  % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
7  quat = zeros(4,1);
8  end
9
10 function R = quatToRotMat(q)
11 % Input: quaternion [w x y z]
12 % Output: corresponding rotation matrix
13
14 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
15 R = zeros(3);
16 end
17
18 function q = rotMatToQuat(R)
19 % Input: rotation matrix
20 % Output: corresponding quaternion [w x y z]
21
22 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
23 q = zeros(4,1);
24 end
25
26 function q_AC = quatMult(q_AB,q_BC)
27 % Input: two quaternions to be multiplied
28 % Output: output of the multiplication
29
30 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION
31 q_AC = zeros(4,1);
32 end
33
34 function B_r = rotVecWithQuat(q_BA,A_r)
35 % Input: the orientation quaternion and the coordinate of the ...
36 %         vector to be mapped
37 % Output: the coordinates of the vector in the target frame
38 % PLACEHOLDER FOR OUTPUT -> REPLACE WITH SOLUTION

```

```

39 B_r = zeros(3,1);
40 end

```

Solution 2.6

$$\mathbf{C}_{IE}(\mathbf{q}) = \mathbf{C}_{I0} \cdot \left(\prod_{k=1}^6 \mathbf{C}_{k-1,k} \right) \cdot \mathbf{C}_{6E} \quad (16)$$

$$\xi(\mathbf{q}) = \frac{1}{2} \begin{pmatrix} \sqrt{c_{11} + c_{22} + c_{33} + 1} \\ \text{sgn}(c_{32} - c_{23}) \sqrt{c_{11} - c_{22} - c_{33} + 1} \\ \text{sgn}(c_{13} - c_{31}) \sqrt{c_{22} - c_{33} - c_{11} + 1} \\ \text{sgn}(c_{21} - c_{12}) \sqrt{c_{33} - c_{11} - c_{22} + 1} \end{pmatrix} \quad (17)$$

where $c_{ij} = \mathbf{C}(i, j)$.

```

1 function quat = jointToQuat(q)
2     % Input: joint angles
3     % Output: quaternion representing the orientation of the end-effector
4     % q_IE.
5     C = jointToRotMat(q);
6     quat = rotMatToQuat(C);
7 end
8
9 function C = quatToRotMat(quat)
10    % Input: quaternion [w x y z]
11    % Output: corresponding rotation matrix
12
13    % Extract the scalar part.
14    quat_w = quat(1);
15
16    % Extract the vector part.
17    quat_n = quat(2:4);
18
19    % Map the unit quaternion to a rotation matrix.
20    C = (2*quat_w^2-1)*eye(3) + 2.0*quat_w*skewMatrix(quat_n) + ...
        2.0*(quat_n*quat_n');
21 end
22
23 function quat = rotMatToQuat(C)
24    % Input: rotation matrix
25    % Output: corresponding quaternion [w x y z]
26    quat = 0.5*[sqrt((1+trace(C)));
27               sign(C(3,2)-C(2,3)) * sqrt(C(1,1) - C(2,2) - C(3,3) + 1);
28               sign(C(1,3)-C(3,1)) * sqrt(C(2,2) - C(3,3) - C(1,1) + 1);
29               sign(C(2,1)-C(1,2)) * sqrt(C(3,3) - C(1,1) - C(2,2) + ...
30               1)];
31 end
32 function quat_AC = quatMult(quat_AB, quat_BC)
33    % Input: two quaternions to be multiplied
34    % Output: output of the multiplication
35    q = quat_AB;
36    p = quat_BC;
37
38    q_w = q(1); q_n = q(2:4);
39    p_w = p(1); p_n = p(2:4);
40
41    skew_q_n = [0      -q_n(3)  q_n(2);
42               q_n(3)  0      -q_n(1);
43               -q_n(2)  q_n(1)  0];
44
45    quat_AC = [q_w*p_w - q_n'*p_n;
46              q_w*p_n + p_w*q_n + skew_q_n*p_n];
47 end

```

```

48
49 function A_r = rotVecWithQuat(quat_AB,B_r)
50     % Input: the orientation quaternion and the coordinate of the ...
           vector to be mapped
51     % Output: the coordinates of the vector in the target frame
52     C_AB = quatToRotMat(quat_AB);
53     A_r = C_AB*B_r;
54 end

```